Apache Mahout QDP: GPU-Accelerated Quantum Data Processing

Guan-Ming Chiu
National Taiwan University
Taipei, Taiwan
gmchiu@arbor.ee.ntu.edu.tw

ABSTRACT

Quantum machine learning (QML) pipelines require encoding classical data into quantum states before any computation can proceed. Existing frameworks such as PennyLane and Qiskit perform this encoding on the CPU, one vector at a time, creating a severe data loading bottleneck that dominates end-to-end training time. We demonstrate **QDP** (Quantum Data Processing), a GPU-accelerated data processing engine built with Rust and CUDA as part of Apache Mahout. QDP provides a complete pipeline from columnar storage formats (Parquet, Arrow) through normalization and quantum state encoding to zero-copy tensor output via DLPack, achieving orders-of-magnitude speedups over existing frameworks. Our interactive demonstration allows users to (1) explore the end-to-end encoding pipeline with configurable parameters, (2) compare latency and throughput against PennyLane and Qiskit in real time, and (3) analyze performance scaling across qubit counts.

PVLDB Reference Format:
Guan-Ming Chiu. Apache Mahout QDP: GPU-Accelerated Quantum Data Processing. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/apache/mahout.

1 INTRODUCTION

Quantum machine learning (QML) combines quantum computing with classical machine learning to explore potential computational advantages [? ?]. A critical step in every QML pipeline is *quantum data encoding*: transforming classical feature vectors into quantum states that can be processed by parameterized quantum circuits. Common encoding strategies include amplitude encoding, angle encoding, basis encoding, and IQP-style encoding [?].

In current practice, frameworks such as PennyLane [?] and Qiskit [?] implement encoding by constructing and simulating a quantum circuit for each input vector. Even with GPU-accelerated backends (e.g., Lightning.GPU, Aer GPU), the encoding step remains sequential and per-vector, creating a bottleneck that grows with qubit count. For a 16-qubit system, each state vector has $2^{16} = 65{,}536$ complex amplitudes; encoding thousands of training samples at this scale can take minutes with existing tools, while the subsequent quantum circuit simulation on GPU completes in seconds. This *data starvation* problem, where the GPU sits idle waiting for encoded data, is analogous to the classical ML data loading bottleneck that motivated systems like PyTorch DataLoader [?], but has received almost no attention in the quantum computing community.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

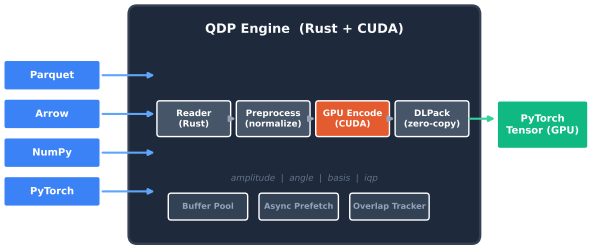

Figure 1: QDP pipeline architecture. Classical data flows from columnar storage through Rust-based readers and preprocessing into CUDA-accelerated encoding kernels, producing GPU-resident quantum state tensors via DLPack.

We present **QDP** (Quantum Data Processing), a GPU-accelerated quantum data encoding engine that is part of Apache Mahout. QDP addresses the data loading bottleneck with three key design decisions:

(1) **Rust + CUDA core.** The entire pipeline, from file I/O through encoding to GPU memory placement, is implemented in Rust with custom CUDA kernels, eliminating Python overhead and GIL contention.
(2) **Columnar format integration.** QDP reads directly from Parquet [?] and Arrow IPC [?] files using native Rust readers, avoiding redundant serialization and deserialization.
(3) **Zero-copy output.** Encoded quantum states are exposed to Python via DLPack [?], enabling zero-copy transfer to PyTorch tensors on GPU without any data movement.

Together, these design choices yield **up to three orders of magnitude** speedup in data-to-state latency compared to PennyLane and Qiskit, as we demonstrate in our interactive system.

2 SYSTEM OVERVIEW

?? shows the QDP pipeline architecture. The system consists of four stages connected by an asynchronous buffer pool.

Reader. The reader stage ingests data from multiple formats: Apache Parquet, Arrow IPC, NumPy `.npy`, PyTorch `.pt`, and TensorFlow TFRecord. For Parquet and Arrow, QDP uses native Rust implementations (via the `arrow-rs` and `parquet` crates) that decode columnar data directly into contiguous memory buffers, avoiding Python-based deserialization. A *streaming* mode for Parquet reads row groups incrementally, enabling processing of datasets

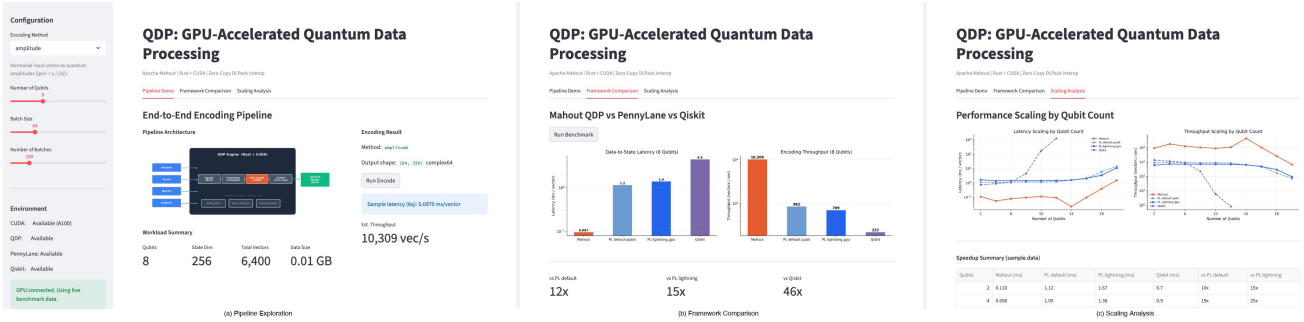

Figure 2: QDP interactive demonstration. (a) Pipeline exploration with configurable encoding parameters and live execution. (b) Framework comparison showing latency and throughput bar charts against PennyLane and Qiskit. (c) Scaling analysis across qubit counts with log-scale performance curves.

larger than available memory. QDP also supports remote object storage URLs (`s3://`, `gs://`) via the `object_store` crate, allowing direct ingestion from cloud data lakes without intermediate downloads.

Preprocessing. Raw feature vectors are normalized according to the requirements of each encoding method. For amplitude encoding, this involves L2 normalization to ensure $\sum_i |a_i|^2 = 1$. Preprocessing runs on the CPU using multi-threaded Rust code and writes results into pinned host memory for efficient DMA transfer to the GPU.

GPU Encoding. Existing frameworks implement encoding by simulating a quantum circuit through sequential gate applications to a state vector. QDP instead provides fused CUDA kernels that directly compute the encoded state vector from classical input data, bypassing circuit simulation entirely and supporting native batching across multiple samples in one kernel launch. QDP supports four methods: *amplitude* (normalize as quantum amplitudes), *angle* (map features to rotation angles), *basis* (encode integer as $|k\rangle$), and *IQP* (entangling interactions between qubits).

Zero-Copy Output. Encoded complex-valued state vectors remain in GPU memory and are wrapped in a DLPack capsule [**?**]. Python consumers call `torch.from_dlpack()` to obtain a PyTorch tensor backed by the same GPU allocation, with no host-device copy or memory duplication. Ownership is managed through DLPack's reference-counted deleter protocol, ensuring the GPU buffer is freed only after all consumers release it.

2.1 Pipeline Infrastructure

Three subsystems maximize GPU utilization:

Dual-stream pipeline. QDP allocates two CUDA streams: one for host-to-device memory copies and one for compute kernels. A CUDA event synchronizes the streams: the copy stream signals completion via `cudaEventRecord`, and the compute stream waits via `cudaStreamWaitEvent` before launching the encoding kernel. This double-buffered design ensures that batch i+1 is being transferred while batch i is being encoded, hiding transfer latency behind compute.

Pinned buffer pool. Repeated `cudaHostAlloc`/`cudaFreeHost` calls are expensive (each triggers a kernel-mode transition). QDP pre-allocates a fixed-size pool of page-locked host buffers at initialization. Buffers are checked out via RAII handles (`PinnedBufferHandle`) and automatically returned on drop, amortizing allocation cost across all batches. A condition variable blocks producers when all buffers are in use, providing natural backpressure.

Overlap tracker. An instrumentation layer records timestamps for copy and compute events on each batch, computing the fraction of time that both streams are active simultaneously. This metric, exposed through the Python API, helps users identify whether their workload is transfer-bound or compute-bound and tune batch sizes accordingly.

2.2 Programming Interface

QDP exposes two Python APIs. The **one-shot API** encodes an entire file into a single GPU tensor, suitable for small datasets that fit in memory:

```
import qumat.qdp as qdp
engine = qdp.QdpEngine(device_id=0)
qt = engine.encode("data.parquet", num_qubits=16,
                   encoding_method="amplitude")
tensor = torch.from_dlpack(qt) # (N, 65536) complex64
```

The **iterator API** provides DataLoader-style batching with a builder pattern, streaming encoded batches for large-scale training loops:

```
loader = (qdp.QuantumDataLoader(device_id=0)
         .qubits(16).encoding("amplitude")
         .batches(100, size=64)
         .source_file("data.parquet"))
for batch in loader:
    state = torch.from_dlpack(batch) # (64, 65536)
```

The iterator internally drives the dual-stream pipeline (§2.1), releasing the Python GIL during Rust/CUDA execution so that model training can proceed concurrently. Input sources include local files (Parquet, Arrow, NumPy, PyTorch, TFRecord), in-memory Python lists, NumPy arrays, PyTorch tensors, and remote URLs (`s3://`, `gs://`). A `QdpBenchmark` builder provides one-call throughput and latency measurement without requiring manual timing code.

3 DEMONSTRATION

We provide an interactive web-based demonstration built with Streamlit that showcases QDP's capabilities through three scenarios. The demonstration runs on a machine equipped with an NVIDIA GPU and allows attendees to interact with the system in real time.

3.1 Scenario 1: Pipeline Exploration

In the first scenario, attendees explore the end-to-end encoding pipeline (**??**a). The interface displays the pipeline architecture and allows users to configure:

- **Encoding method**: amplitude, angle, basis, or IQP.
- **Qubit count**: adjustable from 2 to 20 via a slider, dynamically updating the state vector dimension (2^n) and estimated data size.
- **Batch parameters**: batch size (1–256) and number of batches (10–500).

Clicking "Run Encode" triggers a *real-time* encoding pass through the QDP engine on the attached GPU, displaying the measured per-vector latency and aggregate throughput. Attendees can change parameters and re-run instantly, observing how encoding cost varies with qubit count and method. This scenario demonstrates that QDP handles diverse encoding methods with sub-millisecond per-vector latency.

3.2 Scenario 2: Framework Comparison

The second scenario provides a live head-to-head comparison between QDP, PennyLane, and Qiskit (**??**b). Clicking "Run Benchmark" executes all three frameworks on the same workload in real time, with progress indicators as each framework completes, and dynamically renders:

- **Latency comparison**: a bar chart of milliseconds per vector (log scale), showing QDP's advantage.
- **Throughput comparison**: vectors per second for each framework.
- **Speedup metrics**: computed as the ratio of competitor latency to QDP latency (up to 1,000× depending on encoding and qubit count).

Attendees can adjust the qubit count and encoding method to observe how the performance gap widens at higher qubit counts, where PennyLane and Qiskit's per-vector overhead grows exponentially while QDP's GPU-parallel approach scales gracefully.

3.3 Scenario 3: Scaling Analysis

The third scenario presents scaling curves across qubit counts from 4 to 20 (**??**c). Two line charts show latency and throughput trends for all three frameworks on a logarithmic scale. A summary table provides exact speedup ratios at each qubit count. This scenario allows attendees to understand the asymptotic behavior: QDP maintains practical throughput (>3,000 vectors/sec) even at 20 qubits (2^{20} = 1,048,576 amplitudes), while PennyLane and Qiskit drop below 3 vectors/sec at the same scale.

4 EVALUATION

We evaluate QDP on an NVIDIA A100 GPU (80 GB) with amplitude, angle, and basis encoding across 2–20 qubits. We compare against PennyLane `default.qubit`, PennyLane `lightning.gpu` (CUDA-accelerated), and Qiskit (`initialize()` for amplitude, `RY`/`X` gates for angle/basis via `Statevector.from_instruction`). All measurements include GPU transfer time.

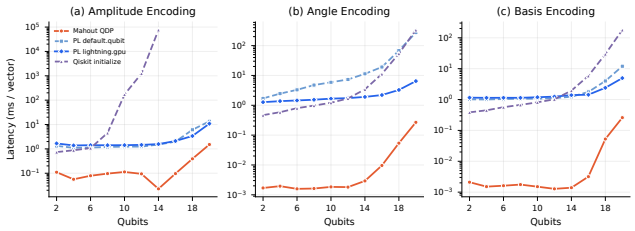

Figure 3: Data-to-state latency scaling (ms/vector, log scale) for amplitude, angle, and basis encoding across four frameworks. QDP (orange) achieves orders-of-magnitude lower latency than both CPU and GPU backends of existing frameworks.

Scaling. **??** shows latency scaling across qubit counts for all three encoding methods. QDP consistently achieves the lowest latency across all configurations, even against PennyLane's GPU-accelerated `lightning.gpu` backend. For angle encoding at 20 qubits, QDP (0.27 ms) is 1,018× faster than PennyLane `default.qubit` (274 ms) and 1,247× faster than Qiskit (335 ms). Even compared to `lightning.gpu` (6.5 ms at 20 qubits), QDP achieves 24× lower latency. Qiskit's circuit-based amplitude encoding (`initialize()`) grows exponentially, reaching 171 ms at 10 qubits while QDP remains at 0.11 ms.

IQP encoding. QDP implements a Fast Walsh-Hadamard Transform (FWT) optimization for IQP encoding, reducing complexity from $O(4^n)$ to $O(n \cdot 2^n)$. Batch encoding with FWT achieves 135× speedup over the naive kernel at 10 qubits, and scales to 20 qubits at 12.3 ms/vector.

5 RELATED WORK

Quantum data encoding. Schuld and Petruccione [**?**] provide a comprehensive survey of encoding strategies for QML. Amplitude encoding is the most commonly used method for variational classifiers, as it can represent 2^n features with n qubits, but requires $O(2^n)$ operations for state preparation. Angle and basis encodings trade expressiveness for simpler circuit constructions [**?**]. Preskill [**?**] discusses the challenges of the NISQ era, where limited qubit counts and noise make efficient classical preprocessing even more critical for practical QML workloads.

Quantum software frameworks. PennyLane [**?**] provides automatic differentiation for hybrid quantum-classical programs and supports multiple backend simulators. Qiskit [**?**] offers a comprehensive toolchain for circuit construction, transpilation, and execution on IBM hardware. Both treat data encoding as a preprocessing step without GPU acceleration, constructing states one vector at a time on the CPU. Neither framework batches encoding or leverages GPU parallelism for state preparation. QuClassi [**?**] demonstrated hybrid quantum-classical architectures at MLSys but did not address the data loading bottleneck. Q2O [**?**] applied quantum optimization to database query planning at VLDB 2025, demonstrating growing interest in quantum-data systems in the database community.

Classical data loading systems. The data starvation problem is well-studied in classical ML: PyTorch DataLoader [**?**] introduced multi-process prefetching and pinned memory transfers, NVIDIA DALI provides GPU-accelerated image decoding, and tf.data pipelines overlap I/O with computation. QDP brings analogous design principles to the quantum data encoding domain, including asynchronous prefetching, pinned buffer pooling, dual-stream pipelining, and zero-copy tensor output. While these techniques are mature in the classical ML ecosystem, to our knowledge QDP is the first system to apply them to quantum state encoding.

6 CONCLUSION

We demonstrated QDP, a GPU-accelerated quantum data processing engine that addresses the data loading bottleneck in QML pipelines. By implementing the full path from columnar storage to GPU-resident quantum states in Rust and CUDA, QDP achieves up to three orders of magnitude lower latency than PennyLane and Qiskit, and 24× speedup over PennyLane's GPU-accelerated `lightning.gpu` backend at 20 qubits. Key engineering contributions include dual-stream pipelining for overlapping host-to-device transfers with GPU encoding, a pinned buffer pool with RAII-based lifecycle management, and zero-copy DLPack interop with PyTorch.

QDP is open source as part of Apache Mahout and supports cloud-native workflows through streaming Parquet ingestion from S3 and GCS. Future directions include: (1) multi-GPU encoding with NCCL-based distribution for large-scale datasets; (2) AMD ROCm/HIP backend for vendor-neutral GPU acceleration; (3) broader cloud data lake support including Azure Blob Storage, MinIO, and Delta Lake/Iceberg table formats; (4) parameterized circuit encoding where gate parameters depend on input data; and (5) integration with quantum hardware backends for direct state preparation on NISQ devices.

ACKNOWLEDGMENTS

This work is part of the Apache Mahout project. We thank the Apache Software Foundation and the Mahout community for their support.